Berquier Raphaël

Programmation système

Question 1

a)

> cat /proc/sys/kernel/pid_max
4194304

b)

> cat /proc/sys/kernel/threads-max
96318

c)

> ps -p 1 -o pid,user,etime,stat,comm
  PID USER   ELAPSED STAT COMMAND
    1 root   11-23:02:11 Ss   systemd

d)

C’est le processus init (le parent de tous les autres processus). Il se stiue : / systemd /sbin/init ou /lib/systemd/systemd

Question 2

Question 3

a)

Le CFS (Completely Fair Scheduler) est le planificateur moderne qui a succédé à l'ancien algorithme O(1).

b)

L'indice nice est une valeur qui détermine la priorité d'exécution d'un processus.

c)

Les appels système permettant de gérer cette priorité sont : nice(), getpriority() et setpriority().

d)

nice : pour lancer une nouvelle commande avec l'indice nice voulu .
renice: pour modifier l'indice nice d'un processus déjà en cours d'exécution.

e)

Sans privilège on peut changer la valeur *nice* entre 0 et 19, pour aller dans les valeurs négatives il faut cette fois avoir les privilèges.

Question 4

a)

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {

   printf("Je suis le processus n°%d\n", getpid());
   printf("Mon père est le processus n°%d\n", getppid());
```

```
    return 0;
}
```

b)

Le processus père ici est l'interpréteur de commande utilisé par le système, puisque c'est lui qui se charge de le lancer.

c)

Si le PID du programme change à chaque exécution avec l'attribution d'un nouveau numéro, le PID du père (le shell) reste quant à lui identique si on relance le programme depuis le même terminal.

Question 5

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {

   pid_t pidFils;
   pidFils = fork();

   switch (pidFils) {
       case -1:
           perror("Échec création d'un nouveau processus ");
           exit(EXIT_FAILURE);

       case 0: // Enfant
           printf("Enfant\n");
           printf("Je suis le processus n°%d\n", getpid());
           printf("Mon père est le processus n°%d\n", getppid());
           exit(0); /* Fin normale du fils */

       default: // Parent
```

```
            printf("Original\n");
            printf("Je suis le processus n°%d\n", getpid());
            printf("Mon père est le processus n°%d\n", getppid());
            printf("Je suis le père du processus n°%d\n", pidFils);
            sleep(1); //Pour un affichage plus propre
            break;

    }

    return 0;
}
```

Question 6

Il y a aura en tout 8 processus puisque :
Le premier programme arrive au premier fork, il y a désormais 2 processus.
Ces deux processus arrivent au deuxième fork, il y a désormais 4 processus.
Ensuite 8 pour le 3eme fork.

Question 7

a)

Accessible via la commande man 3 sleep dans le Volume 3, la fonction sleep n'est pas un appel système direct mais une fonction de la bibliothèque C (POSIX) qui utilise l'appel système nanosleep() (Volume 2)

b)

L'instruction sleep(10) met en pause le processus pendant dix secondes, le faisant passer en mode « sommeil » pour rendre la main à l'ordonnanceur.

Question 8

C'est pourquoi, lorsqu'on lance le programme en arrière-plan avec le symbole &, la commande ps -l affiche le processus avec la lettre S (Sleeping) : il est alors mis en pause par le noyau et ne consomme aucun temps CPU en attendant la fin du chronomètre.

Question 9

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

int main(void) {
   pid_t pidFils;
   int status;

   pidFils = fork();

   if (pidFils == -1) {
       perror("Échec de fork");
       exit(EXIT_FAILURE);
   }

   if (pidFils == 0) {
       // CODe lorsque c'est l'enfant qui execute
       printf("Je suis le processus n°%d\n", getpid());
       printf("Mon père est le processus n°%d\n", getppid());

       char *argv[12];
       argv[0] = "./testExecve";
       argv[11] = NULL;          // marquage de fin de tableau

       srand(time(NULL));
```

```
        for (int i = 0; i < 10; i++) {
            int val = rand() % 21;

            argv[i + 1] = malloc(3 * sizeof(char));

            sprintf(argv[i + 1], "%d", val);
        }

        // Execution du code
        execve("./testExecve", argv, NULL);

    } else {
        // Code lorsque c'est le parent qui execute
        printf("Je suis le processus n°%d\n", getpid());
        printf("Mon père est le processus n°%d\n", getppid());
        printf("Je suis le père du processus n°%d\n", pidFils);

        // Le père attend que le fils se termine et récupère la somme via
la valeur retourné lors de la fin du programme
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Code de terminaison du processus n°%d : %d\n", pidFils,
WEXITSTATUS(status));
        }
    }

    return 0;
}
```